

Machine Learning

Mikkel Moth Billing
202307989

2. februar 2026

Indhold

Indhold	1
1 Linear Models	3
1.1 Decision Trees	4
Entropy	5
Loss Function	6
1.2 Error	7
Loss Function Error	7
1.3 Linear Regression	8
1.4 Logistical Regression	9
1.5 Multinomial Regression	10
2 Non-Linear Models	12
2.1 Growth Function	12
2.2 Testing	13
2.3 Overfitting	13
Regularization	13
Validation	14
2.4 Cross Entropy Loss Function	15
3 Neural Nets	15
3.1 Back Propagation	15
3.2 Overfitting	16
3.3 Convolution	17
4 Bias-Variance	18
4.1 Bagging	19
Random Forest	19
Out-of-Bag Samples	20
5 Boosting	20
5.1 Ada Boost	20
5.2 Gradient Boosting	21

6	Support Vector Machines	22
6.1	Convex Optimization	23
6.2	Kernels	24
6.3	Regularization	25
7	Unsupervised Learning	25
7.1	Cluster Analysis	25
8	Dimension Reduction	26
8.1	Principal Component Analysis	26
8.2	Autoencoders	27
8.3	Random Projections	28
9	Word Embedding	29
9.1	Bag of Words	29
9.2	TF-IDF	29
9.3	Hashing	30
9.4	CBOW	30
9.5	Skip-Gram	31

1 Linear Models

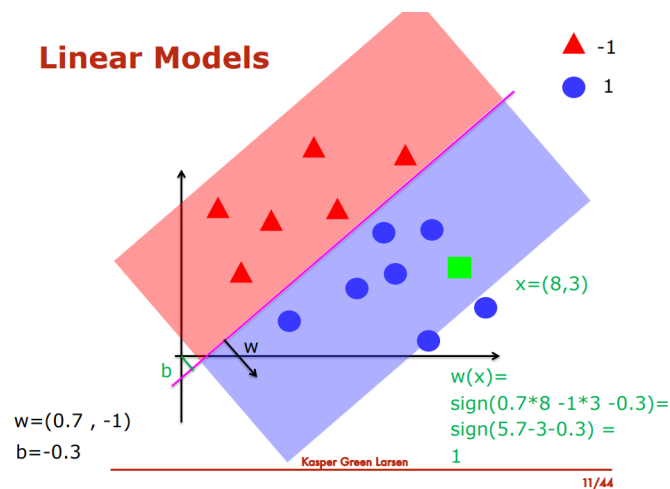
Hypothesis consists of the features vector:

$$x = \{x_1, \dots, x_d\}$$

and a bias b . if the linear combination of the weights and the input, and compare it to the bias, we can make predictions.

Example of mortgage, approve is 1 and decline is -1 . So we can predict that we approve

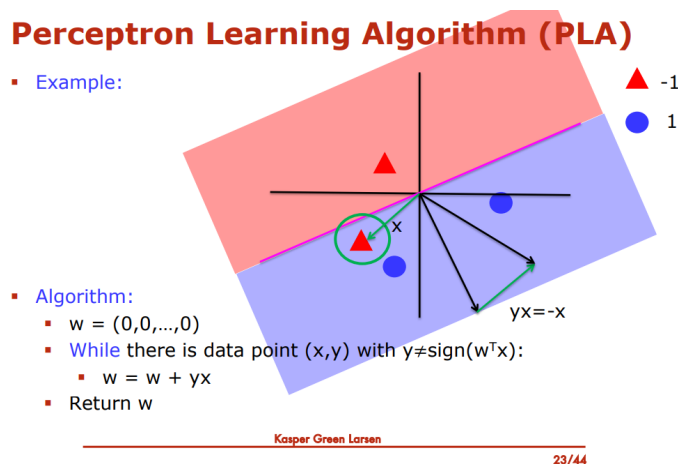
$$w_1x_1 + \dots + w_dx_d > -b$$



To find the parameters we can use **Perceptron Learning Algorithm (PLA)**.

Algorithm: While there is a data point (x, y) with

$$y \neq \text{sign}(w^T x) \rightarrow w = w + yx.$$



As we can see the algorithm can enter an infinite loop, as there might not be a linearly separable dataset.

A way to fix this is to use Pocket PLA which saves the most accurate hyperplane.

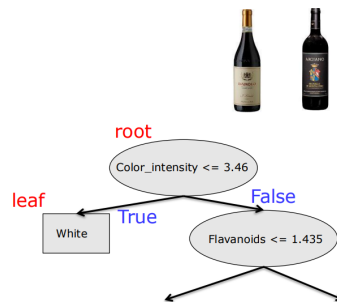
Pocket PLA

- Would like to still do something useful if not linearly separable!
- Pocket version: Remember best w in your pocket.
- Algorithm:
 - $w = (0,0,...,0)$
 - $bestScore = 0$
 - $bestW = w$
 - While you have more time, let (x,y) have $y \neq \text{sign}(w^T x)$:
 - $w = w + yx$
 - $score = \text{percentage of data correctly classified by } w$.
 - If $score > bestScore$:
 - $bestW = w$, $bestScore = score$
 - Return $bestW$

1.1 Decision Trees

Decision Trees

- Classifying wine.
 - Red, White, Rosé
- Predict(x):
 - Start at root node.
 - Test condition in root and descent to relevant child.
 - Upon reach leaf:
 - Return value stored at leaf.
- Node:
 - Question tests **one** feature against **one** value.

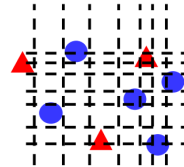
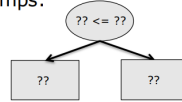


We would like to keep the decision tree small to not overfit to the training data. So we could find the best performing tree with k nodes. But this is **NP-hard**, so we rely on **heuristics**. With a greedy learning algorithm with $k = 2$:

Small Decision Trees

- Greedy Learning Algorithm.

- Decision Stumps:



- Find best decision stump:

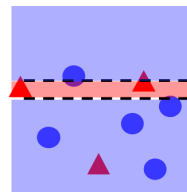
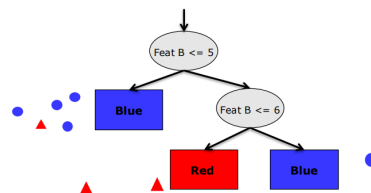
- For each of d features:
 - For each of $n-1$ relevant splits:
 - Try that root, partition points into leaves, assign best leaves, compute accuracy.
- Use best split found.

We test the tree on all possible split, this has $O(n^2d)$ runtime.
 Another way to build a tree is to increase k to maximise the accuracy.

Small Decision Trees

- Final Tree:

$k=2$ example



Accuracy: 7/8

Entropy

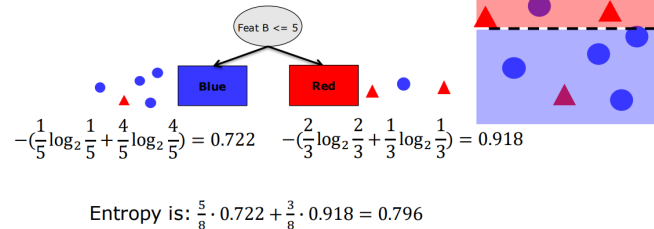
Another way to test decision tree is to base it on entropy. If we have n point with n_i as the numbers of points with label i . And $p_i = n/n_i$. Then entropy is:

$$H(S) = - \sum_{i=1}^k p_i \log_2 p_i.$$

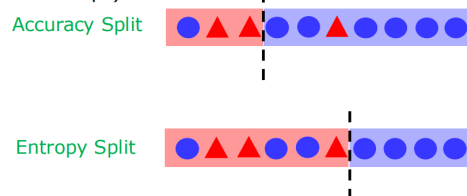
Entropy is a measure of uncertainty in distributions, high entropy is high uncertainty. $H(S)$ describes the expected number of bits it takes to describe the label of a random element drawn from S . If all elements in S are equal $H(S) = 0$. If there is an equal amount $H(S) = \log_2 k$.

Entropy

- Entropy of tree: $\sum_v \frac{|S_v|}{n} H(S_v)$



- May give different trees than using accuracy.
- 1-dimensional example, i.e. one feature.
 - Entropy prefers "pure" splits over accuracy.
 - Seem to work best in practice when constructing decision trees (larger than stumps).



Loss Function

A Loss Function weighs how much a mistake influences a model. We can then try to minimize the loss function over the training points. An example is the Least Square Loss Function:

$$L(y', y) = (y' - y)^2.$$

Small Regression Trees

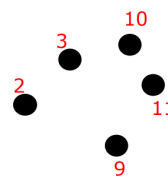
- Greedy Learning Algorithm.
 - Start in simple case:
 - Our tree may only contain 1 node.
 - Root is a leaf.



- What is best choice of root?
 - The x that minimizes $\frac{1}{n} \sum_{i=1}^n (x - y_i)^2$ is the mean:
 - $x = \frac{1}{n} \sum_{i=1}^n y_i$.

Mean is $(2+3+9+10+11)/5=35/5=7$

Loss is $((2-7)^2 + (3-7)^2 + (9-7)^2 + (10-7)^2 + (11-7)^2)/5=14$.



1.2 Error

In-Sample Error (E_{in}) is the error on the training data, and is what we try to minimize. The Out-Sample Error (E_{out}) is the error on the unknown data, this is what we wan't minimized. So we hope that $E_{in} \approx E_{out}$.

Loss Function Error

For the speiel Loss function 0-1 Loss, we know the error is

$$E_{in}(h) = \frac{1}{n} \sum_{i=1}^n 1_{h(x_i) \neq f(x_i)}.$$

The Out-Sample error is just the probabilty that $h(x) \neq f(x)$.

The Hoeffding inequality says that the probability of the two errors being farther appart then some number ε is

$$P(|E_{in}(h) - E_{out}(h)| > \varepsilon) \leq 2e^{-2\varepsilon^2 n},$$

where n is training examples. If we have multiple hypothesis M we get:

$$P(|E_{in}(h) - E_{out}(h)| > \varepsilon) \leq 2Me^{-2\varepsilon^2 n}.$$

From this we can prove that minimizing $E_{in}(h)$ is a good strategy since for any other h' in H , we get that $E_{out}(h) \leq 2\varepsilon + E_{out}(h')$, so any other h' is at most 2ε better then h .

We can also use Hoeffding inequality to find out what sampling size we should use. If we want the difference in error to be less then 0.1 with 95% confidence, and $M = 1000$, we get:

$$2000e^{-2 \cdot 0.1^2 n} < 0.05 \Rightarrow n > 50 \ln(40.000) = 530,$$
$$n > \frac{\ln(M/(1-c))}{2\varepsilon^2}.$$

If we are using an algorithm with infinite hypothesis, we can use a large number for M , for hyperplanes representable using d floats, that would be $M = 2^{32d}$.

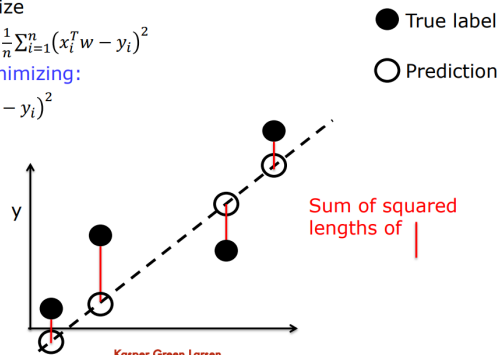
1.3 Linear Regression

The Trick

- **Regression:**
 - Features: $x = (x_1, \dots, x_d)$
 - Labels: R
- **Linear Model:**
 - Parameters of hypothesis w : $(w_1, w_2, \dots, w_d, b)$
 - **Predict:** $w(x) = w_1x_1 + w_2x_2 + \dots + w_dx_d + b = x^T w + b$
- **Trick:**
 - Annoying to have special bias parameter b .
 - Make data $(d+1)$ -dimensional and set first coordinate of all data points equal to 1.
 - Then parameters $(w_0, w_1, w_2, \dots, w_d)$.
 - Predict $w(x) = w_0 \cdot 1 + w_1x_1 + w_2x_2 + \dots + w_dx_d = x^T w$.

Linear Regression

- **Goal:** Minimize
 - $E_{in}(w) = \frac{1}{n} \sum_{i=1}^n (x_i^T w - y_i)^2$
- **Same as minimizing:**
 - $\sum_{i=1}^n (x_i^T w - y_i)^2$



Derivative of Matrix

If $l = (l_1, \dots, l_k)$ is a function from $R^d \rightarrow R_k$, then the Jacobian is a matrix of dimension $k \times d$.

$$\frac{\partial l}{\partial x} = \begin{pmatrix} \frac{\partial}{\partial x_1} l_1(x) & \dots & \frac{\partial}{\partial x_d} l_1(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} l_k(x) & \dots & \frac{\partial}{\partial x_d} l_k(x) \end{pmatrix}$$

Some helpful tricks are:

$$\frac{\partial}{\partial x} Ax + b = A,$$

$$\frac{\partial}{\partial x} x^T x = 2x^T.$$

Linear Regression

- **Goal:** Minimize
 - $l(w) = (Xw - y)^T (Xw - y) = \sum_{i=1}^n (x_i^T w - y_i)^2$
- **Compute gradient as transpose of matrix of derivatives.**
- **Matrix of derivatives:**
 - $\frac{\partial}{\partial w} (Xw - y)^T (Xw - y) =$
- **Using Chain rule:**
 - $2(Xw - y)^T \frac{\partial}{\partial w} (Xw - y) =$
 - $2(Xw - y)^T X =$
 - $2(w^T X^T X - y^T X).$
- **Gradient as transpose:**
 - $2(w^T X^T X - y^T X)^T = 2(X^T X w - X^T y).$

Rules:

- $\frac{\partial}{\partial x} Ax + b = A$
- $\frac{\partial}{\partial x} x^T x = 2x^T$

Same as we got earlier

$$\Rightarrow w = (X^T X)^{-1} X^T y$$

1.4 Logistical Regression

If we want to calculate a probability based on some data, we use logistical regression. To map a regression to a number between 0 and 1, we use the Sigmoid Function,

$$\theta(z) = \frac{1}{1 + e^{-z}}.$$

So for our prediction function we use:

$$\theta(w^T x) = \frac{1}{1 + e^{-w^T x}}.$$

So what we want is for $\theta(w^T x)$ to be equal to $P[y = 1|x]$, the likelihood of seeing y_i given x_i is $\theta(y_i w^T x_i)$. Then the total likelihood of a data set (S) is

$$P[S|w, X] = \prod_{i=1}^n \frac{1}{1 + e^{-y_i w^T x_i}},$$

which we can then maximise to get our prediction function. We do this by minimizing the average of the negativ log function:

$$\begin{aligned} E_{in}(w) &= -\frac{1}{n} \ln(P[S|w, X]) = \frac{1}{n} \sum_{i=1}^n \ln(1 + e^{-y_i w^T x_i}), \\ \nabla E_{in}(w) &= -\frac{1}{n} \sum_{i=1}^n \frac{1}{1 + e^{-y_i w^T x_i}} e^{-y_i w^T x_i} y_i x_i \\ &= \frac{1}{n} \sum_{i=1}^n -y_i x_i \theta(-y_i w^T x_i). \end{aligned}$$

But we can not set this function to 0 to minimize, so we use Gradient Decent, which is to "roll a ball down a hill", by guessing $w = w - \eta \nabla l(w)$, where η is a step size (usually $\eta = 0.1$) and $\nabla l(w)$ is the gradient of the function. But this can be slow, so we could use Stochastic Gradient Decent, be just taking a batch of datapoints,

$$\begin{aligned} g &= \frac{1}{B} \sum_{i=1}^B -y'_i x'_i \theta(-y'_i w^T x'_i), \\ w &= w - \eta g. \end{aligned}$$

1.5 Multinomial Regression

If we have k different classes that we need to fit for instead of just 1 and -1 . We can use One-in-K Encoding, this means that we use unit vector (instead of just numbers), to classify different targets. For example wine we could have:

Red: (1, 0, 0)
White: (0, 1, 0)
Rose: (0, 0, 1)

So now our label vector turn into a $(n \times K)$ matrix, and our weight vector turn into a $(d \times K)$ matrix. To reduce numbers in a vector to a probability distrubtion we use the softmax function which scales the sum of a vector to 1

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_{i=1}^K e^{z_i}}.$$

Which for code implementation could also be calculated as:

$$\text{softmax}(z)_j = e^{z_j - z_a - \ln(\sum_{i=1}^K e^{z_i - e^{z_a}})},$$

where z_a is $\max(z)$

Then for Multinomial Regression our prediction of X $\text{softmax}(XW)$, where we apply the softmax function row-wise. Then the probabilty that we see the correct labels using W is

$$P[S|X, W] = \prod_{i=1}^n y_i^T \text{softmax}(x_i^T W),$$

which we can rewrite to negative log and then minimize:

Multinomial Regression

- K classes, n data points $x_1, \dots, x_n$:

$$X = \begin{pmatrix} x_{1,1} & \dots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \dots & x_{n,d} \end{pmatrix}, Y = \begin{pmatrix} y_{1,1} & \dots & y_{1,K} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \dots & y_{n,K} \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \dots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \dots & w_{d,K} \end{pmatrix}$$

- Maximize over W

$$\prod_{i=1}^n y_i^T \text{softmax}(x_i^T W)$$

- Negative log likelihood. Scale by 1/n. Min

$$E_{in}(W) = -\frac{1}{n} \sum_{i=1}^n \ln(y_i^T \text{softmax}(x_i^T W))$$

- Luckily, we are told that the gradient is

$$\nabla E_{in}(W) = -\frac{1}{n} X^T (Y - \text{softmax}(XW))$$

Sanity check:

Gradient should have same shape as W

Entry (i,j) equals $\partial E_{in}(W) / \partial w_{i,j}$

Then we can do Gradient Decent og Stochastic Gradient Decent

Gradient Descent

- Gradient Descent for Multinomial Regression:

- Initialize W (e.g. random $N(0,1)$ distributed entries).

- While more time:

$$\nabla E_{in}(W) = -\frac{1}{n} X^T (Y - \text{softmax}(XW))$$

$$W = W - \eta \nabla E_{in}(W)$$

- Return W.

Stochastic Gradient Descent

- Mini-batch SGD for Multinomial Regression:

- Initialize W (e.g. random $N(0,1)$ distributed entries).

- While more time:

- Permute $(x_1, y_1), \dots, (x_n, y_n)$ randomly.

- For $i=1, \dots, n/B$:

- Let $(x'_1, y'_1), \dots, (x'_B, y'_B)$ be next B data items.

$$G = -\frac{1}{B} X'^T (Y' - \text{softmax}(X'W))$$

$$W = W - \eta G$$

- Return W.

- Each time we permute is called an **epoch**.

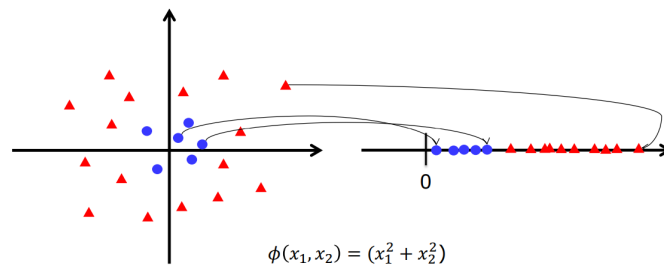
- **Practical:** Reduce η by a small factor after fixed number of epochs.

2 Non-Linear Models

A way to work with data that can not be trained to a hyperplane, like for example a circle, is to transform the feature vector to a new dimension

$$\begin{aligned}\varphi: \mathbb{R}^d &\rightarrow \mathbb{R}^{d'} \\ \varphi(x_1, x_2) &= (x_1^2 + x_2^2).\end{aligned}$$

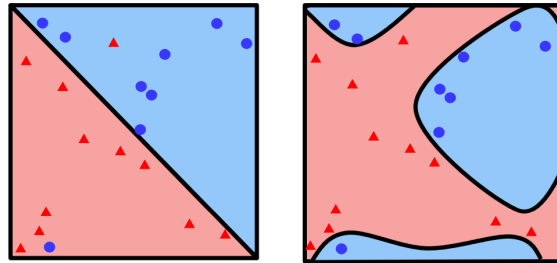
You can then use linear models of the new model.



This can make more accurate models, but if they are too complex they might be more wrong.

Some Considerations

4th degree transform
(1, x₁, x₂, ..., x₁³x₂, x₁⁴, ...)



- Not worth high complexity to decrease E_{in} a tiny bit. Probably large E_{out}

2.1 Growth Function

The Growth Function states that there is a finite number of hypothesis sets, dependent on the number of points in the set

$$m_H(n) = \max_{x_1, \dots, x_n \in X} |\{(h(x_1), \dots, h(x_n)) : h \in H\}|.$$

We can relate these hypothesis sets to M in Hoeffding's inequality

$$P(|E_{in}(h) - E_{out}(h)| > \epsilon) \leq 8m_H(2n)e^{-\epsilon^2 n/8}.$$

But it can be hard to find a function for $m_H(n)$, so instead we can bound the growth function with Shattering. If we have k points in a 2D space, we

need to be able to all 2^k dichotomies, meaning that we can create all subsets of points with hypothesis H . (See ML 15 video) If all the dichotomies can be generated the points are shattered. If there is no data set of k points that can be shattered then k is a break point for H .

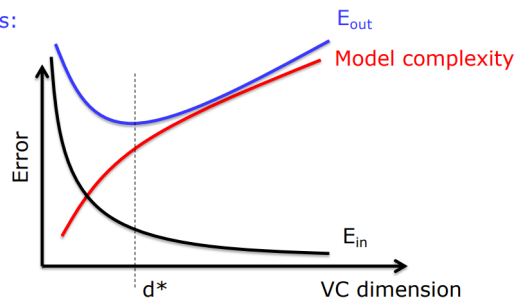
This lead to the Vapnik-Chervonenkis dimension (VC dimension), which is d such that $d = k - 1$. From this we know that $m_H(n) \leq n^d + 1$, then

$$P(|E_{in}(h) - E_{out}(h)| > \varepsilon) \leq 8((2n)^d + 1)e^{-\varepsilon^2 n/8}.$$

Here we can see that given enough data we can always get that E_{in} and E_{out} are close, if the VC dimension is bounded ($d < \infty$).

VC Dimension

- H with VC dimension d has generalization:
 - $\Pr[|E_{in}(h^*) - E_{out}(h^*)| > \varepsilon] \leq 8((2n)^d + 1)e^{-\varepsilon^2 n/8}$
- Conclusions:



2.2 Testing

2.3 Overfitting

Overfitting is when we have a hypothesis that is too complex, this means we get a low in-sample error, but high out-sample error. To solve this we use hyperparameters λ , which is user given parameters that changes something about the learning algorithm. Some examples are:

Regularization

To make sure we don't overfit, we can use regularization. This could be linear regression with weight decay, where we add a limit to the size of the weights. Here we minimize

$$\begin{aligned} & \frac{1}{n} \sum_{i=1}^n (w^T x_i - y_i)^2 + \lambda \|w\|^2, \\ \nabla_w &= \frac{1}{n} (2X^T X w - 2X^T y) + 2\lambda w, \\ \Rightarrow w &= (X^T X + n\lambda I)^{-1} X^T y. \end{aligned}$$

Regularization

- Linear Regression with Weight Decay

- Minimize

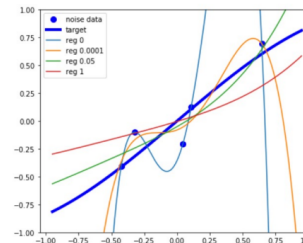
- $$\frac{1}{n} \sum_{i=1}^n (w^T x_i - y_i)^2 + \lambda \|w\|^2$$

- Does regularization help?

- For linear models, it often does.
- Depends a lot on model.

- For linear regression, even has its own name:

Ridge Regression



Validation

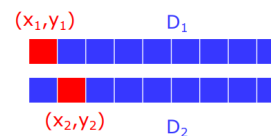
Here we take k inputs from the training data, to use for validation instead. We can then evaluate a $E_{val} \approx E_{out}$, so the learning algorithm can make a decision based on this. We could run the algorithm with different λ , and evaluate all the hypothesis, and then pick the best function. A trick is that after we find λ then we can retain with all the data.

We can also do cross validation to test the hypothesis. Here we train the algorithm on all the data except one point, and then evaluate on that one point. We do this for all points and average the loss over all points. In practice doing it on one point at the time is too slow, so you could split the data in 5, and do it 5 times.

Cross Validation

- Cross validation with $k=1$:

- For each x_i of data items $x_1, \dots, x_n$:
 - Create $D_i = \{x_1, \dots, x_n\} \setminus \{x_i\}$.
 - Train on D_i .
 - Compute validation loss e_i on (x_i, y_i) .
- Set $E_{cval} = (1/n)(e_1 + e_2 + \dots + e_n)$.



- Model Selection:

- Do above for every choice of (hyper-)parameters.

- Observations:

- For each i , use almost all data. Great.
- For each i , validate on only one point. Bad.
- Hope: Averaging gives good estimate. Cannot prove. Not always true.

2.4 Cross Entropy Loss Function

The Cross Entropy Loss function is a generalization of negative log likelihood, that can be used for any machine learning model h that outputs K probabilities $p_1, \dots, p_K$ summing to 1.

$$L(h(x), y) = - \sum_{j=1}^K y_j \ln(h(x)_j),$$

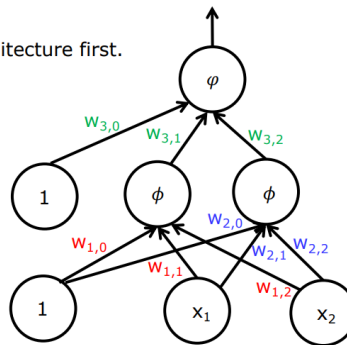
$$E_{in}(h) = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^K y_{i,j} \ln(h(x_i)_j).$$

3 Neural Nets

When building a neural network we first have to look at each neuron. The neuron takes in weights and data, and determines what it should send out to other connected neurons. An activation function φ , it could run could be $\text{ReLU} = \max(0, w^t x)$, or the Sigmoid function.

Neural Net Architectures

- **Feed Forward Neural Net.**
 - When train, choose architecture first.
 - Nodes
 - Edges
 - Activation Functions
 - Then train weights.



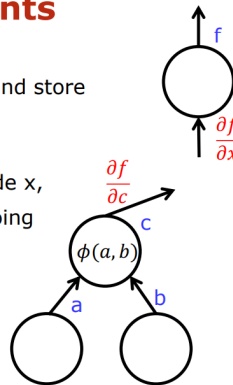
When running a neural network we first decide the architecture, and then the weights gets updated. A simple architecture is fully connected feed forward neural network as seen in the figure above.

3.1 Back Propagation

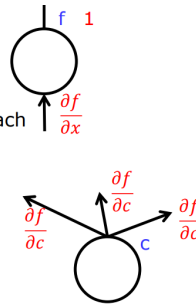
If we see the neural net as a function $f(x_1, \dots, x_i)$, where $x_1, \dots, x_i$ all have values, we want to find the numerical value of the partial derivative of the function, with respect to all the parameters. We do this by using the chain rule on all the nodes, and finding their partial derivative.

Computing Gradients

- **General Algorithm:**
 - Forward pass, compute and store values of all nodes.
- **Backprop:**
 - Compute $\frac{\partial f}{\partial x}$ for every node x , starting from top and going down.
 - To compute $\frac{\partial f}{\partial a}$:
 - $\frac{\partial f}{\partial a} = \frac{\partial f}{\partial c} \frac{\partial c}{\partial a} = \frac{\partial f}{\partial c} \frac{\partial \phi(a, b)}{\partial a}$
 - Already computed $\frac{\partial f}{\partial c}$
 - $\phi(a, b)$ is simple and a, b known from forward pass.



- **Special Rule:**
 - If a node sends output to multiple destinations:
 - Backprop computes a gradient on each edge.
 - These must be summed to obtain actual $\frac{\partial f}{\partial c}$ to be used further down the computation graph.



Because we can find the derivatives we can do stochastic gradient decent. This can be done with pytorch.

3.2 Overfitting

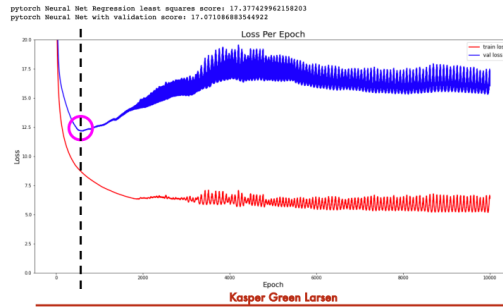
To avoid overfitting in neural nets, we can again use validaion. We then store the best performing parameters. We can add to this, by stopping if the parameters do not improve over som time.

Validation – Early Stopping

- Run 10.000 epochs of SGD:
- Use Validation set and keep best:

$$E_{in} = 6.2 \quad E_{val} = 12.5, E_{in} = 8.5$$

$$E_{out} \approx E_{test} = 17.4 \quad E_{out} \approx E_{test} = 17.1$$



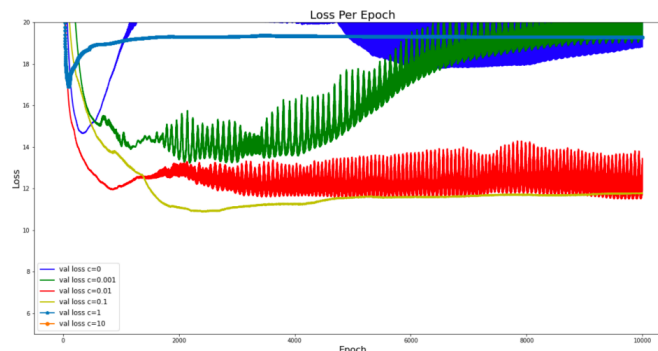
4/10

We can also do weight decay where we add λ to the weight (not the bias).

$$E_{in}(h) = \frac{1}{n} \sum_{i=1}^n L(h(x_i), y) + \lambda \sum_{w \in h} w^2.$$

Regularization

Remember: Include weight decay in loss when training, but validation must compute raw loss (no weight decay).



3.3 Convolution

For image recognition we can use matrix convolution. If we have a grayscale image, or matrix with values between 0 and 1. We can apply a convolution matrix to it, to get a matrix with edges. The weight in the convolution matrix can be trained with back propagation. If we don't have a grayscale picture, we can split the picture in red, green and blue. Then we add up the edges, so we only get one edge matrix. This is done multiple times with different convolution matrixes, to get multiple channels that can be used to identify pictures.

You can then use Pooling which could be taking the max or minimum of a selected area, to shrink the picture.

Convolutions

- Formally:

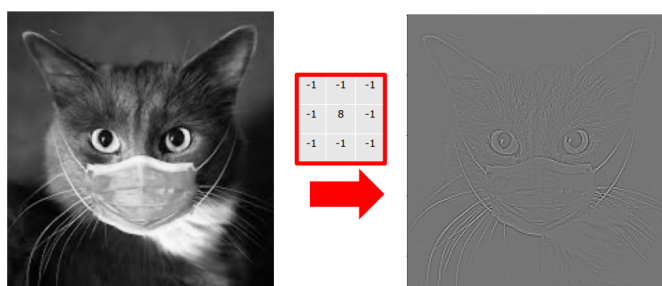
- $k \times k$ convolution specified by a $k \times k$ matrix. Often k is odd.
- Convolution: "Slide" convolution matrix across input.

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

-1	-1	-1
-1	8	-1
-1	-1	-1

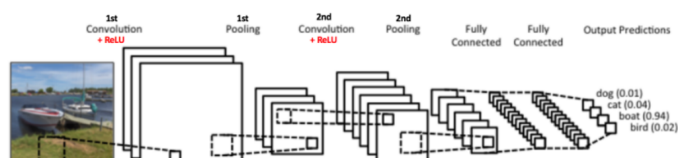
Result:
New image with
"higher-level"
features. Edges
rather than
individual colors.

0.9	-1	-1	-2	0.1	-7
0.1	...	...	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...
...	...	...	...	...	4.7



Deep Nets for Images

- Repeat a block multiple times
- End with fully connected (and softmax for classification)



4 Bias-Variance

The goal of Bias Variance is to say something about the out of sample error like with VC dimensions, but this time we want to take our hypothesis into account. Here we find the bounding by

$$\mathbb{E}_D[E_{out}(h^D)] = \mathbb{E}_x[\mathbb{E}_D[(h^D(x) - \bar{h}(x))^2]] + \mathbb{E}_x[(\bar{h}(x) - f(x))^2],$$

where $\bar{h}(x)$ is the average value of all the hypothesis we have trained

$$\bar{h}(x) = \mathbb{E}_D[h^D](x) = \frac{1}{m} \sum_i h^{D_i}(x).$$

We can note that $\mathbb{E}_x[\mathbb{E}_D[(h^D(x) - \bar{h}(x))^2]]$ is a measure of how far the hypothesis is of from the average hypothesis, also called the variance. $\mathbb{E}_x[(\bar{h}(x) - f(x))^2]$ is then the bias, thereby the name.

4.1 Bagging

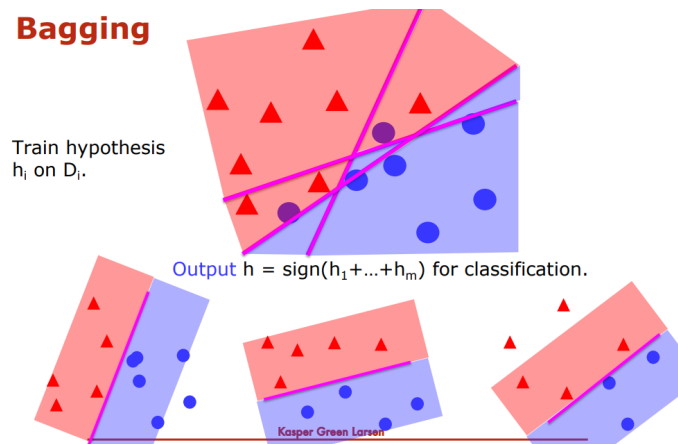
Bootstrap Aggregating (bagging) is a technique to reduce the variance in a returned hypothesis. This means that we average some independent variables which reduces variance. $X_1, \dots, X_m$ with variance σ^2 , then $Y = (X_1 + \dots + X_m)/m$ has variance σ^2/m .

Bagging:

- Do m times:
 - Sample $n' < n$ data points from D (with replacement). Call set D_i .
 - Train hypothesis h_i on D_i .
- Output $h = (1/m)(h_1 + \dots + h_m)$ for regression.
- Output $h = \text{sign}(h_1 + \dots + h_m)$ for classification.

Observations:

- Reduces risk of overfitting noise on a point.
- Should reduce variance.

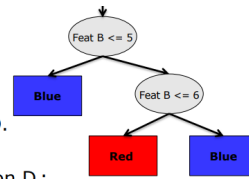


Random Forest

This model is used for decision and logistic trees, as a more accurate form of bagging, since the datasets are not independent, because it is the same set we are training on. If there is pairwise correlation $\rho > 0$, the variance of Y is $\rho\sigma^2 + (1 - \rho)\sigma^2/m$. So we also need to reduce the correlation.

Random Forest

- **Random Forest:**
 - Do m times:
 - Sample $n' < n$ data points from D .
 - Call set D_i .
 - Train decision/regression tree T_i on D_i :
 - Each time computing a split:
 - Randomly choose p of the d data features.
 - Only allow splitting based on these p features.
 - **Output** $h = (1/m)(T_1 + \dots + T_m)$ for regression.
 - **Output** $h = \text{sign}(T_1 + \dots + T_m)$ for classification.
- **Suggestions:**
 - Classification: $p = \sqrt{d}$, Regression: $p = d/3$.
 - **Or: Grid Search!**



Kasper Green Larsen

Out-of-Bag Samples

In Bagging and Random Forest, when we pick data from sets, we can use the data not picked for validation.

5 Boosting

5.1 Ada Boost

We can boost a weak learner to a strong learner. A weak learner is a classifier that performs slightly better than guessing. A Strong learner is a classifier that performs perfectly. We do this by training a lot of weak learners and combining them. If we have a weighted dataset with a distribution D over X , we can reweight the dataset, which is basically what bagging does.

We run an algorithm (A) on a weighted training set (T) to get hypothesis set $h_t \in H$ and compute weights α_t . Then we return a voting classifier

$$f(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right).$$

AdaBoost

- **Input:** Training data $(x_1, y_1), \dots, (x_n, y_n)$. Base learning algorithm A.
- **Initialize:** $D_1(i) = 1/n$ for all i .
- **Algorithm:**
 - For $t=1, \dots, T$:
 - Run A to minimize weighted 0-1 loss L_{D_t} . Outputs h_t .
 - $er_t = L_{D_t}(h_t) = \sum_{i=1}^n D_t(i) 1_{h_t(x_i) \neq y_i}$.
 - $\alpha_t = \frac{1}{2} \ln \left(\frac{1 - er_t}{er_t} \right)$.
 - $D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$ where $Z_t = \sum_{j=1}^n D_t(j) \exp(-\alpha_t y_j h_t(x_j))$.
- **Output:** $f(x) = \text{sign}(\sum_{t=1}^T \alpha_t h_t(x))$

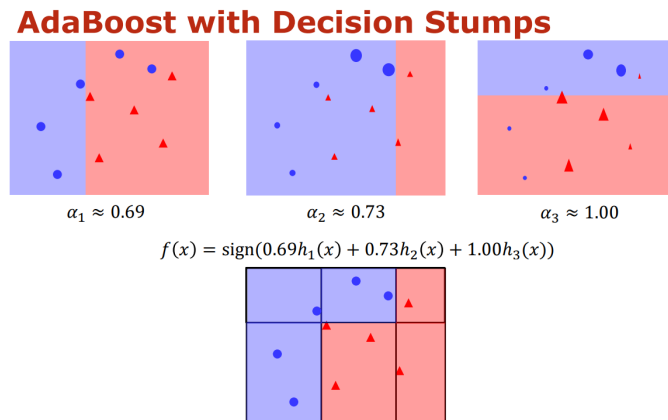
Start with
uniform weighing

What this does is put more emphasis on points that are misclassified, so the algorithm will focus more on these points. $D_{t+1}(i)$ can also be calculated as

$$\begin{aligned} D_{t+1}(i) &= \frac{\frac{1}{n} \exp\left(\sum_{j=1}^t -\alpha_j y_i h_j(x_i)\right)}{\prod_{j=1}^t Z_j}, \\ &= \frac{\frac{1}{n} \exp(-y_i F_t(x_i))}{\prod_{j=1}^t Z_j}. \end{aligned}$$

If T becomes big enough the in sample error will eventually become 0, meaning we have a strong learner. this happens if $T > \ln(n)/2\gamma^2$, where $\gamma = 1/2 - \varepsilon$, and ε is the error of the weak learner.

An example for a simple decision stump can be seen in figure below.



5.2 Gradient Boosting

In Gradient Boosting we first train h_1 on some dataset and then calculate the loss, we then train a new hypothesis on the difference between the predicted value, and the actual value, and combine the two hypothesis.

Gradient Boosting

- **Regression:**

- $f(x) = \sum_{t=1}^T \alpha_t h_t(x)$
- $E_{in}(f) = \frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2$

- **Very natural algorithm:**

- Train regression tree h_1 to minimize least squares loss.
- Train h_2 with data $(x_1, y_1 - h_1(x_1)), \dots, (x_5, y_5 - h_1(x_5))$.

Example data:

$(x_1, 500)$
 $(x_2, 650)$
 $(x_3, 800)$
 $(x_4, 900)$
 $(x_5, 1150)$

Mistakes $y_i - h_1(x_i)$:

500-450 = 50
 650-500 = 150
 800-1000 = -200
 900-950 = -50
 1150-1000 = 150

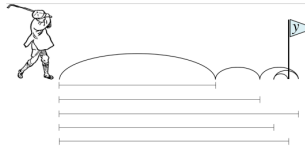
Example predictions:

$h_2(x_1)=80$
 $h_2(x_2)=120$
 $h_2(x_3)=-200$
 $h_2(x_4)=0$
 $h_2(x_5)=100$

Mistakes $y_i - f(x_i)$:

500-450-80 = -30
 650-500-120 = 30
 800-1000+200 = 0
 900-950-0 = -50
 1150-1000-100 = 50

Gradient Boosting



- **Very natural algorithm:**
 - Let $h_1(x) = \frac{1}{n} \sum_{i=1}^n y_i$
 - Let $\alpha_1 = 1$.
 - For $t=2, \dots, T$:
 - Train regression tree h_t to minimize least squares loss with data $(x_i, y_i - \sum_{j=1}^{t-1} \alpha_j h_j(x_i))$.
 - Let $\alpha_t = \eta$.
 - **Output** $f(x) = \sum_{t=1}^T \alpha_t h_t(x)$

This can be done for any pointwise loss function, not just least square loss. The loss function $L(x, y)$ needs to be differentiable, then we get the final general algorithm for gradient Boosting

Gradient Boosting

- **Gradient Boosting:**
 - Let $h_1(x) = c$ where c minimizes $E_{in} = \frac{1}{n} \sum_{i=1}^n L(c, y_i)$. Let $\alpha_1 = 1$.
 - For $t=2, \dots, T$:
 - Train regression tree h_t to minimize least squares loss with data $X, \hat{y} = - \begin{pmatrix} \frac{\partial L(F_{t-1}(x_1), y_1)}{\partial F_{t-1}(x_1)} \\ \vdots \\ \frac{\partial L(F_{t-1}(x_n), y_n)}{\partial F_{t-1}(x_n)} \end{pmatrix}$.
 - For each leaf l in h_t :
 - Let l predict value c_l minimizing $\sum_{i \in l} L(F_{t-1}(x_i) + c_l, y_i)$
 - Let $\alpha_t = \eta$.
 - **Output** $f(x) = \sum_{t=1}^T \alpha_t h_t(x)$

Here $F_{t-1}(x_n)$ is the current prediction. No matter what the loss function is we still minimize least squares loss.

6 Support Vector Machines

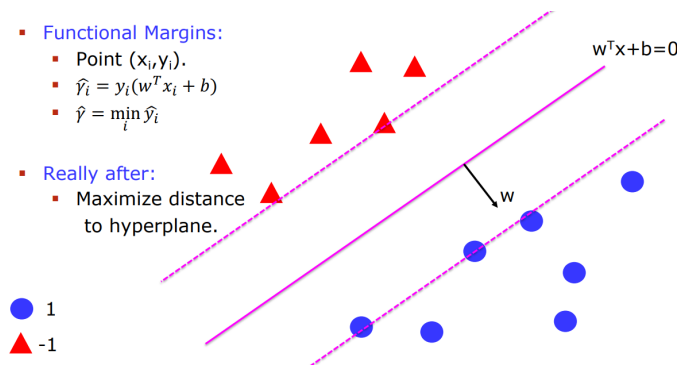
Support Vector Machines is an expansion of linear classification. We can use this when we have a large boundry between classes, to give a margin instead of just a line. We do this by maximising γ in

$$y_i(w^T x_i + b) \geq \gamma, \quad i = 1, \dots, n$$

$$\|w\| = 1.$$

But this is not simple, so we change the function, with the requirement that it should be convex. We need to minimize $\|w\|^2/2$ as a function of w and b :

$$y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, n$$



6.1 Convex Optimization

The general convex optimization problem is to minimize $f(x)$ subject to $g_i(x) \leq 0$ where f and g_i are convex functions. We do this with the generalized lagrangian:

$$L(x, a) = f(x) + \sum_{i=1}^n a_i g_i(x).$$

Given the constraints this means that we can solve:

$$\min_x \theta_p(x) = \min_x \max_{a: a_i \geq 0} L(x, a).$$

this is the prime value $p^* = \min_x \theta_p(x)$, we also introduce dual value:

$$\begin{aligned} \theta_D(a) &= \min_x L(x, a), \\ d^* &= \max_{a: a_i \geq 0} \theta_D(a), \end{aligned}$$

and observe $d^* \leq p^*$.

Convex Optimization

- **Result:**
 - If f and g_i 's are convex functions and the constraints g_i are strictly feasible (there is an x with $g_i(x) < 0$ for all i), then
 - $d^* = p^*$
 - There is an optimal solution x^*, α^* satisfying:
 - $\frac{\partial L(x^*, \alpha^*)}{\partial x_i} = 0$ for $i=1, \dots, d$
 - $\alpha_i^* g_i(x^*) = 0$ for $i=1, \dots, n$
 - $g_i(x^*) \leq 0$ for $i=1, \dots, n$
 - $\alpha_i^* \geq 0$ for $i=1, \dots, n$
- **Implications:**
 - Can solve dual problem, and, assume solution satisfies all of the above.

To use this for SVM we set:

$$\begin{aligned} f(w, b) &= ||w||^2/2, \\ g_i(w, b) &\leq 0, \\ g_i(w, b) &\leq 1 - y_i(w^T x_i + b). \end{aligned}$$

All these functions are convex, and we can therefore use the dual value.

$$\begin{aligned} \max_{a: a_i \geq 0} \min_{w, b} f(w, b) + \sum_{i=1}^n a_i g_i(w, b), \\ \max_{a: a_i \geq 0} \min_{w, b} \|w\|^2/2 - \sum_{i=1}^n a_i (y_i (w^T x_i + b) - 1). \end{aligned}$$

From KKT conditions we can find that

$$\begin{aligned} w &= \sum_{i=1}^n a_i y_i x_i, \\ \sum_{i=1}^n a_i y_i &= 0. \end{aligned}$$

That gives the final problem:

$$\begin{aligned} \max_{a: a_i \geq 0} \sum_{i=1}^n a_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a_i a_j y_i y_j x_i^T x_j, \\ w &= \sum_{i=1}^n a_i y_i x_i, \\ b &= -\frac{1}{2} \left(\min_{i: y_i=1} w^T x_i + \max_{i: y_i=-1} w^T x_i \right). \end{aligned}$$

6.2 Kernels

Like for the linear classifications we don't need to only apply SVM to linear problems, we can use kernels which is the same as feature transform $\varphi: R^d \rightarrow R^{d'}$, the corresponding kernel would be: $K: R^d \times R^d \rightarrow R$, which would be $K(x, z) = \varphi(x)^T \varphi(z)$.

Kernels

- **Kernel SVM formulation:**
 - $\max_{a: a_i \geq 0} \sum_{i=1}^n a_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a_i a_j y_i y_j K(x_i, x_j)$
 - subject to:
 - $\sum_{i=1}^n a_i y_i = 0$
 - $b = -\frac{1}{2} \left(\min_{i: y_i=1} \sum_{j \neq 0} a_j y_j K(x_i, x_j) + \max_{i: y_i=-1} \sum_{j \neq 0} a_j y_j K(x_i, x_j) \right)$
- **Prediction on x:**
 - $\text{sign}(\sum_{i \neq 0} a_i y_i K(x_i, x) + b)$
- **Running time:** Assume K takes t time to evaluate.
 - Assume k support vectors.
 - $O(n^2 t)$ + normal SVM time to solve. Predict in $O(kt)$.

$O(n^2 t)$ to prepare.

$O(kt)$

$O(kt)$

Polynomial Kernel:

$$K(x, z) = (x^T z + c)^r.$$

Gaussian Kernel:

$$K(x, z) = \exp(-\lambda \|x - z\|^2), \quad \lambda > 0.$$

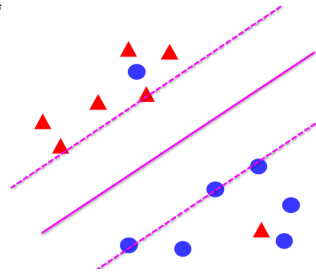
6.3 Regularization

If the data is not linearly separable, we can use regularization, which changes the calculations to:

Regularization

- Regularized SVM:
- $\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j$
- subject to:
 - $\sum_{i=1}^n \alpha_i y_i = 0$
 - $0 \leq \alpha_i \leq C \quad i=1, \dots, n$
- Simply added an upper bound constraint on α_i .
- Still have:

$$w = \sum_{i=1}^n \alpha_i y_i x_i.$$



But here we can not find b the same way since we have outliers.

7 Unsupervised Learning

7.1 Cluster Analysis

Clustering is a way of splitting up data in clusters and is unsupervised learning. Data can be split up in more clusters creating a cluster hierarchy. A way of making these clusters is with k-means, where we make k clusters, defined from a cluster center. We then minimize the loss of this center. For a set of centers C let $\pi(x) \in C$ be the cluster center. Then minimize

$$\text{cost}(S, C, \pi) = \sum_{x \in S} \|x - \pi(x)\|^2.$$

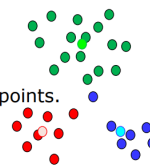
It turns out that the point that reduces the cost is the mean of the points in the cluster.

$$\pi(x) = \frac{1}{|S_i|} \sum_{x \in S_i} x.$$

The most famous k-means algorithm is Lloyd's Algorithm:

Lloyd's Algorithm

- **Lloyd's algorithm:**
 - Pick k random input points and let C equal those points.
 - Repeat until $\text{newCost}/\text{oldCost} > 1-\epsilon$
 - Assign every point to its closest center in C
 - Replace C by the means of each cluster



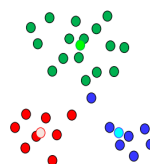
- **Properties:**
 - Always converges since each step reduces cost
 - **Quality of final clustering depends on initialization**
 - May be arbitrarily worse than optimal clustering
 - **May take exponential number of steps to converge**
 - Terminate if improvement in cost less than some ϵ -fraction

Typically not in practice

We can expand on this algorithm to make the initial guess better:

K-Means++ Algorithm

- **K-means++ algorithm:**
 - Let c_1 be a uniform random input point.
 - For $i=2, \dots, k$:
 - Pick c_i to equal input point x with probability
$$p_x = \frac{\min_{j < i} \|x - c_j\|^2}{\sum_{y \in S} \min_{j < i} \|y - c_j\|^2}$$
 - Repeat until convergence:
 - Assign every point to its closest center in C
 - Replace C by the means of each cluster
- Lloyd's algorithm with clever initialization.
 - Can prove: expected cost of solution $O(\log k \text{ OPT})$.
 - $\text{OPT} = \text{cost of optimal clustering}$



8 Dimension Reduction

8.1 Principal Component Analysis

In PCA we project a feature vector onto a k-dimensional space, to reduce features to k. We project x onto a subspace of k unit vectors $z_1, \dots, z_k$. and get a new basis

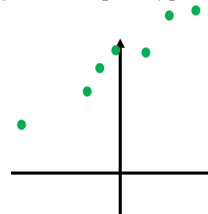
$$\hat{x} = \begin{pmatrix} x^T z_1 \\ \vdots \\ x^T z_k \end{pmatrix}$$

PCA

- **PCA:** Choosing k directions $z_1, \dots, z_k$ for projection **greedily**:
 - For $i=1, \dots, k$:
 - Let z_i be unit length such that:
 - z_i is orthogonal to $z_1, \dots, z_{i-1}$.
 - Sample variance of $x_1^T z_i, \dots, x_n^T z_i$ is largest among all unit vectors z that are orthogonal to $z_1, \dots, z_{i-1}$.

Linear Algebra Fact:

Let $X = \begin{pmatrix} - & x_1 & - \\ \vdots & \vdots & \vdots \\ - & x_n & - \end{pmatrix}$ then z_i equals the eigenvector μ_i corresponding to the i 'th largest eigenvalue of $X^T X$.



To find z_i we minimize the variance:

$$\begin{aligned} \sum_{j=1}^n \left| x_j - \sum_{i=1}^k (x_j^T z_i) z_i \right|^2, \\ = - \sum_{j=1}^n \sum_{i=1}^k (x_j^T z_i)^2, \end{aligned}$$

which is the same as maximizing

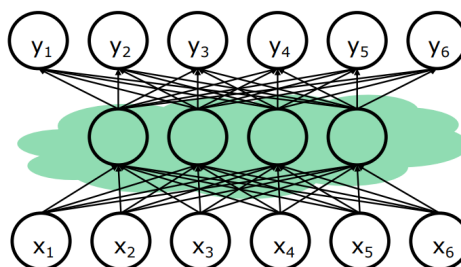
$$\sum_{j=1}^n \sum_{i=1}^k (XZ)_{i,j}^2 = \text{tr}(Z^T X^T X Z).$$

8.2 Autoencoders

Autoencoders are a neural net that reduces the amount of features, by minimizing the loss of a "bottleneck" structure. This is a biased version of PCA, that can also use non linear functions.

Autoencoders

- **Neural Nets for Embeddings.**
 - **Intuition:** If $y=x$ then all information about x must be present in bottleneck layer.
 - **Use** output of bottleneck layer as features.



8.3 Random Projections

Another way to reduce features is Johnson-Lindenstrauss projection. Here we want to preserve the distance between points before and after the feature transform, we do this by making the projection

$$X \rightarrow \sqrt{\frac{d}{k}} XZ,$$

where $z_1, \dots, z_k$ are random unit length orthogonal vectors.

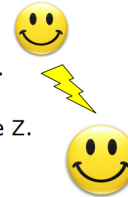
Random Projections

- **Johnson-Lindenstrauss:** Pick $z_1, \dots, z_k$ as random unit length and orthogonal vectors. Project data X to $\sqrt{d/k} XZ$.

$$X = \begin{pmatrix} - & x_1 & - \\ \vdots & \vdots & \vdots \\ - & x_n & - \end{pmatrix}, Z = \begin{pmatrix} | & \dots & | \\ z_1 & \dots & z_k \\ | & \dots & | \end{pmatrix}$$

- **PCA:** Pick $z_1, \dots, z_k$ as the k eigenvectors corresponding to largest eigenvalues of $X^T X$ and project data X to XZ .

- **Johnson-Lindenstrauss:** Preserves distances.
- **PCA:** Guarantees best reconstruction if do XZZ^T .
 - Johnson-Lindenstrauss **does not**.
- **Johnson-Lindenstrauss:** Much easier to compute Z .
 - Z does not depend on X .
 - Can be chosen before seeing data.



Random Projections

- **Johnson-Lindenstrauss Transform**
 - Given n feature vectors $x_1, \dots, x_n$ in $\mathbb{R}^d$.
 - For any precision $0 < \epsilon < 1/2$ let $L = \frac{1}{\sqrt{k}} A$ where A is $k \times d$ with each entry independently $N(0,1)$ distributed and $k = \theta(\epsilon^{-2} \ln n)$.



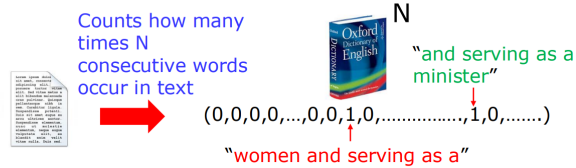
- With probability at least $1-1/n$, it holds for all x_i, x_j :
 - $(1 - \epsilon) \|x_i - x_j\|^2 \leq \|Lx_i - Lx_j\|^2 \leq (1 + \epsilon) \|x_i - x_j\|^2$

9 Word Embedding

9.1 Bag of Words

- **N-Grams:**

- Adding order to words.
- Group together every N consecutive words.
- Feature for every N-tuple of words in dictionary.



- "Am a single Arab women and serving as a minister, there is a limit to my personal income and investment level"

9.2 TF-IDF

TF-IDF

- **Inverse Document Frequency IDF:**

- Some words occur in almost all documents:
 - "the, a, or, and,..."
- Some words occur in few document:
 - "viagra, investment, charity,..."
- For collection of documents $d_1, \dots, d_n$ (think the training documents) and a term t (a word):
 - **Document frequency** (fraction of documents with t)

$$df(t) = \frac{|\{d_i : t \in d_i\}|}{n}$$
 - **Inverse document frequency**

$$idf(t) = \ln\left(\frac{1}{df(t)}\right) = \ln\left(\frac{n}{|\{d_i : t \in d_i\}|}\right)$$



TF-IDF

- **Term-Frequency TF:**

- For document d and term t , occurrences:
 - $f_{t,d} = \# \text{occurrences of } t \text{ in } d$.
- **Idea:** "Normalize" frequencies.
- **Replace** by the term-frequency $tf(t,d) = f_{t,d} / |d|$.



"We shall execute the transaction under a legitimate arrangement without breaking the law"

- $tf(\text{"the"}, d) = 2/13$
- $tf(\text{"under"}, d) = 1/13$.

TF-IDF

- Bag-of-words:
 - Entry t in vector for document d is:
 - $f_{t,d}$
- Tf-idf:
 - Entry t in vector for document d is:
 - $tf(t,d) \cdot idf(t) = \frac{f_{t,d}}{|d|} \cdot \ln\left(\frac{n}{|\{d_i : t \in d_i\}|}\right)$



Words occurring often in document have large weights.

Rare words are weighed higher than common words.

9.3 Hashing

Hash Functions

- Hash Functions
 - Basic idea:
 - Functions that map from large domain $[d]$ to small domain $[k]$.
 - Should distribute items in $[d]$ "evenly" in $[k]$.
 - A bit more formally:
 - Hash functions are *chosen* using randomness.
 - Example: Fix a prime $p > d$.
 - Choose a uniformly in $\{1, \dots, p-1\}$.
 - Choose b uniformly in $\{0, \dots, p-1\}$.
 - Use hash function $h_{a,b}(x) = ((ax+b) \bmod p) \bmod k$.
 - $\Pr_{a,b}[h_{a,b}(x) = h_{a,b}(y)] \leq 1/k$ for any $x \neq y$.



9.4 CBOW

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1, \dots, d_n$:
 - Entry per pair of words (t_1, t_2) . Counts #co-occurrences (t_1, t_2) in $d_1, \dots, d_n$.

	apples	taste	...	good	pears
apples	0	1	...	1	0
taste	1	0	...	2	1
...	...	...	...	...	...
good	1	2	...	0	0
pears	0	1	...	0	0

Co-occurrence matrix

- $d_1 = \text{"apples taste good and are healthy"}$
- $d_2 = \text{"the taste of pears is delicious"}$
- $d_3 = \text{"basil adds a good taste to the dish"}$

CBOW

- Defining a Supervised Learning Problem

- Context window: Specified by a width around a word.
 - Example: Context window of width 2 around "who"

Bootstrap issue:
Need a representation to learn a repr.?

- "partner abroad who can accommodate 200M USD for Investment"

- Learning Task:

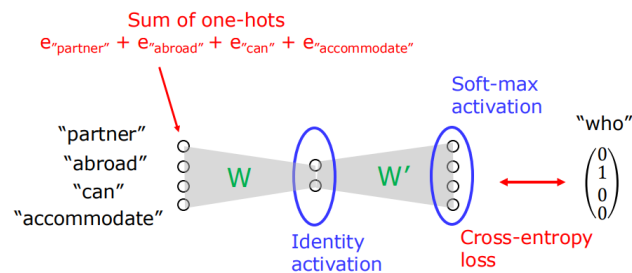
- Predict middle word from context.

- Example:

- Input is ("partner", "abroad", "can", "accommodate")
- Label is "who"

How do we represent these as input to a learning algo?

CBOW (("partner", "abroad", "can", "accommodate"), "who")



Train on all the training documents $d_1, \dots, d_n$.

Mini-batch Stochastic Gradient Descent as always.

9.5 Skip-Gram

Skip-Gram

- Alternative to CBOW

- Very similar.

- Context window: Specified by a width around a word.

- "partner abroad who can accommodate 200M USD for Investment"

context window

- Learning Task:

- Skip-gram: Predict context from middle word.

- CBOW: Predict middle word from context.

- Example:

- Input is "who"
- Label is ("partner", "abroad", "can", "accommodate")

Skip-Gram

- Alternative to CBOW
 - Very similar.
- Context window: Specified by a width around a word.
- *"partner abroad who can accommodate 200M USD for Investment"*
context window
- Learning Task:
 - Skip-gram: Predict context from middle word.
 - ("who", "partner")
 - ("who", "abroad")
 - ("who", "can")
 - ("who", "accommodate")

One training example per
context word